



Overview of the course

A simple exercise - add numbers in parallel

Performance analysis

Amdahl's law

Introduction to Parallel Computing

Course Outline

Architecture

- Multiprocessor architectures
- Memory models – shared, distributed
- Emerging platforms – multicore chips, GPGPUs
- Inter-processor communication

Programming

- Basic concepts
- Programming tools
 - POSIX threads
 - MPI
- Architecture-specific programming
 - CUDA
- Performance analysis

Algorithms

- Basic operations such as
 - Reduction
 - Scatter-gather
 - Prefix-sum
- Sorting algorithms
- Graph algorithms
- Others

Applications

- Monte-Carlo
- Matrix computations
- N-body computations
- Big Data
- Others

+

Overview

Parallel and Distributed Computing

- Fundamental limits on processor speed
- Availability of low cost hardware
- Grand challenge problems



Applications

- Scientific simulation: weather, astrophysics, environment, ...
- Engineering design: aerospace, petroleum, electronics, materials, ...
- Health sciences: medicine, biology, biochemistry, ...
- Finance
- E-commerce: analysis, data mining
- Defense
- Visualization
- Big Data

Adding numbers in parallel

	1																														
	1	2	3	4	5	6	7	8	9	10	1	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
1	7	1	6	6	2	7	9	2	1	6	7	1	9	9	5	8	4	4	2	2	4	1	9	5	2	6	1	3	9	3	141
2	9	7	2	4	6	6	4	3	3	4	6	1	6	3	2	5	3	8	6	8	4	8	7	2	1	9	6	4	8	5	150
3	4	1	8	1	8	7	5	1	2	1	4	4	4	8	9	1	2	4	4	3	8	6	1	1	7	5	4	3	1	1	118
4	2	5	4	9	8	2	3	8	2	2	5	7	6	5	7	5	3	5	9	4	3	3	7	6	1	4	1	6	8	4	144
5	5	5	9	5	9	7	5	3	4	5	7	8	8	9	8	9	5	3	7	2	3	9	7	5	9	7	1	2	5	4	175
6	7	7	3	7	1	9	5	2	9	8	6	7	1	9	5	4	7	9	7	4	8	7	2	5	3	5	6	1	8	2	164
7	2	6	4	4	5	7	4	3	5	4	7	1	8	6	4	7	5	3	9	3	9	2	8	2	8	3	1	9	7	8	154
8	5	9	9	6	4	1	6	1	7	1	9	8	7	8	3	5	2	1	4	2	3	6	2	5	3	6	6	3	7	6	145
9	1	8	6	3	2	5	6	1	2	6	9	1	2	7	5	8	5	5	5	1	2	5	7	7	7	3	1	5	7	8	140
10	2	6	6	6	5	4	9	5	1	7	7	9	6	2	5	7	9	5	1	1	3	1	4	7	1	5	8	3	9	2	146
11	4	3	4	8	9	3	2	5	8	8	8	1	8	5	9	1	3	9	1	2	1	9	2	4	3	1	5	8	7	3	144
12	9	5	6	8	2	7	8	5	9	2	9	7	5	9	9	4	2	1	9	8	7	2	8	3	9	6	8	1	8	6	182
13	6	6	4	7	7	3	9	9	5	6	7	8	3	6	7	7	5	5	4	2	3	6	9	8	5	2	6	9	4	4	172
14	5	6	1	2	7	9	6	8	2	6	7	4	6	1	3	2	6	8	4	4	7	3	6	1	7	7	8	7	4	7	154
15	8	1	4	8	5	1	4	1	4	3	5	7	2	2	9	6	6	5	2	5	6	9	3	3	9	4	9	9	2	3	145
16	3	6	2	7	7	1	3	8	9	7	4	4	8	4	3	5	4	8	7	6	5	5	4	8	1	4	4	4	6	7	154
17	5	8	2	6	2	1	3	5	4	7	4	6	4	3	1	3	3	3	5	5	4	9	5	1	3	9	5	6	5	4	131
18	3	2	4	6	7	7	2	3	6	4	4	9	2	9	1	6	8	8	1	3	5	1	9	3	9	1	4	4	1	9	141
19	6	8	5	2	7	1	9	1	2	5	7	9	5	7	9	4	2	2	6	8	7	2	5	6	1	5	9	1	4	7	152
20	6	6	5	6	6	6	1	5	6	1	6	1	6	7	2	3	9	9	3	6	3	4	8	3	7	6	1	7	5	2	146
21	7	2	9	8	2	7	4	5	4	5	8	1	2	7	9	6	5	7	4	1	5	7	3	7	1	6	3	9	7	3	154
22	6	2	6	1	4	9	2	6	9	2	5	9	8	8	5	3	3	7	2	5	6	5	9	9	4	1	4	6	9	7	162
23	5	9	5	1	8	1	9	3	5	9	8	5	3	1	9	1	1	5	5	2	5	7	3	8	8	5	4	7	6	5	153
24	6	6	7	3	7	2	1	5	2	3	7	3	9	9	7	4	1	5	8	8	9	8	6	9	5	3	1	1	4	2	151
25	9	2	6	4	4	2	8	8	4	2	1	7	1	2	8	7	8	6	9	3	5	6	4	8	2	1	1	1	1	3	133
26	7	7	8	2	2	8	8	5	3	6	8	3	7	9	9	7	5	3	4	2	9	3	6	4	6	1	9	3	4	6	164
27	6	1	6	5	8	4	5	2	2	5	6	4	9	5	5	1	7	2	4	3	5	5	7	2	4	7	7	9	6	4	146
28	2	8	5	2	1	6	7	2	1	2	4	1	8	1	9	2	2	2	7	4	3	2	1	9	9	4	1	6	7	2	120
29	7	8	4	4	6	5	3	5	3	4	2	3	5	6	4	7	7	1	9	6	4	2	9	9	9	3	7	7	3	4	156
30	5	6	1	6	4	3	9	5	5	7	7	2	9	5	6	8	5	6	5	6	2	2	8	4	9	4	2	4	6	8	159
	1																														
	8																														
	159	157	151	147	155	141	159	125	129	138	4	141	167	172	177	146	137	149	153	119	148	145	169	154	153	133	133	148	168	139	4496

Adding numbers in parallel ...

Approach

- Divide task into subtasks (Partitioning)
- Map subtasks to processors (Scheduling)
- Distribute data to processors (distributed memory architecture only)

+

Algorithm

Step 1: Each processor adds numbers to obtain its partial sum

Step 2: Processors collaborate to compute sum of partial sums

Issues

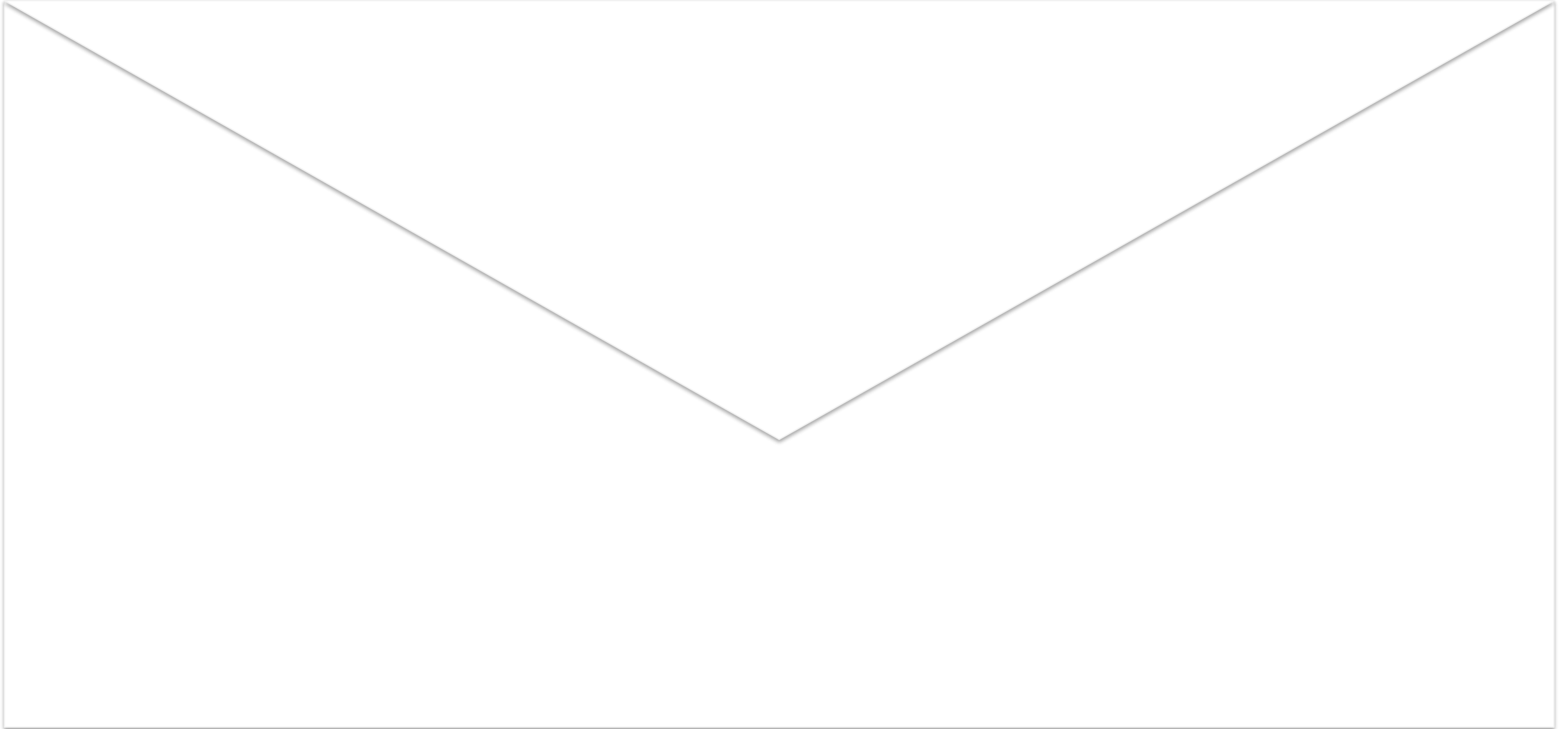
- Load Balancing: Reduce the difference in task sizes to balance load among processors
- Granularity: Select best task size
 - *Fine grain*: Many small tasks for better load balance, increases overhead of task scheduling
 - *Coarse grain*: Fewer large tasks, equal size, lower overhead in scheduling
- Architectural Implications
 - *Distributed memory*
 - A subtask's data should be placed on same processor
 - Requires inter-processor communication in Step 2
 - Near-neighbor communication to minimize time
 - Map binary tree on processors to compute sum in $\log p$ steps
 - *Shared memory*
 - Distinct memory locations to store partial sums
 - Smart work distribution in Step 2 to compute sum in $\log p$ steps
 - Synchronization among processors is required before and during Step 2

Performance Analysis

- T_1 : execution time on single processor
- T_p : execution time on p processors
- Speedup: $S_p = T_1 / T_p$
- Efficiency: $E_p = S_p / p$

+

Speedup and efficiency of the parallel number adder



Performance of parallel number adder

- One processor adds partial sum

+

$$T_p = \frac{n}{p} + p$$

$$S_p = \frac{p}{1 + \frac{p^2}{n}}$$

$$E_p = \frac{1}{1 + \frac{p^2}{n}}$$

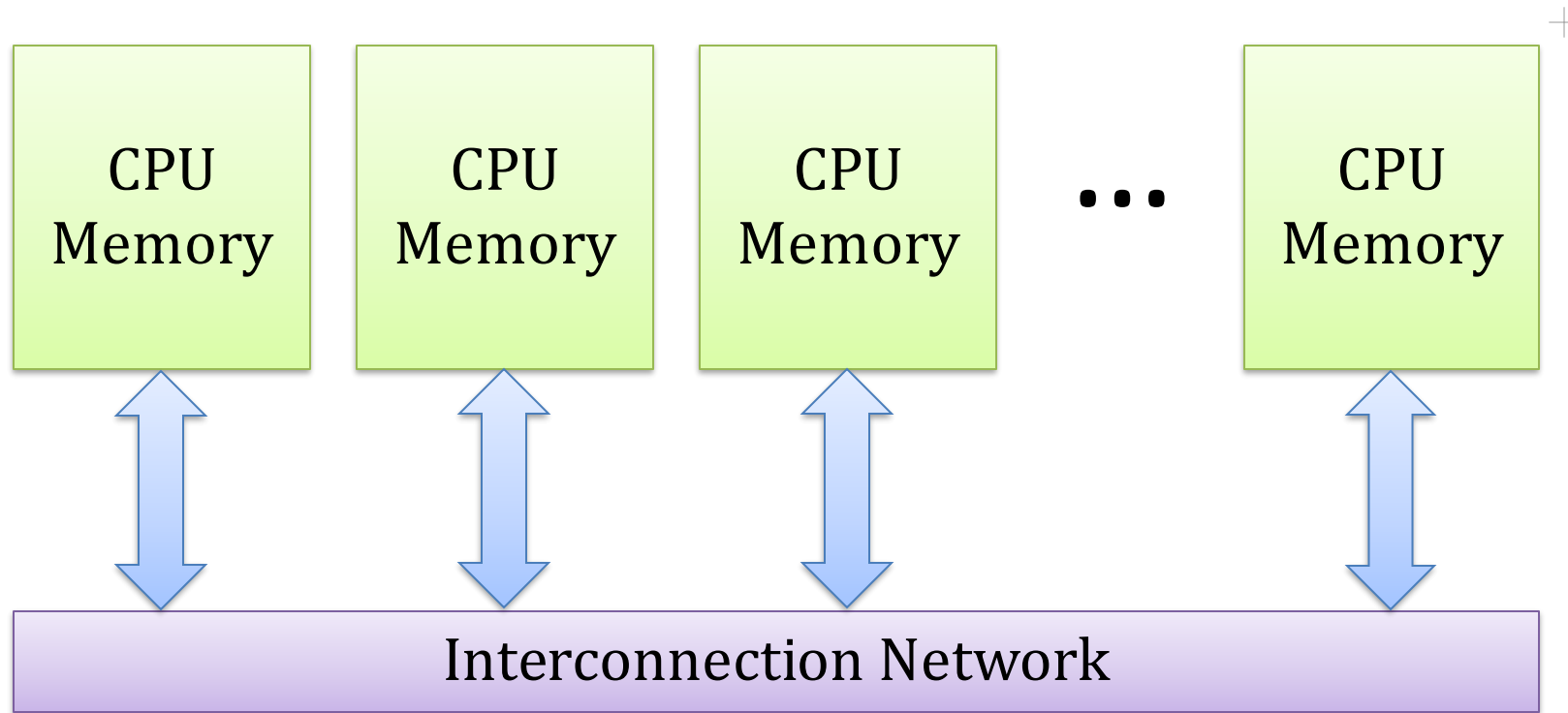
- Partial sums added in a binary tree style

$$T_p = \frac{n}{p} + \log p$$

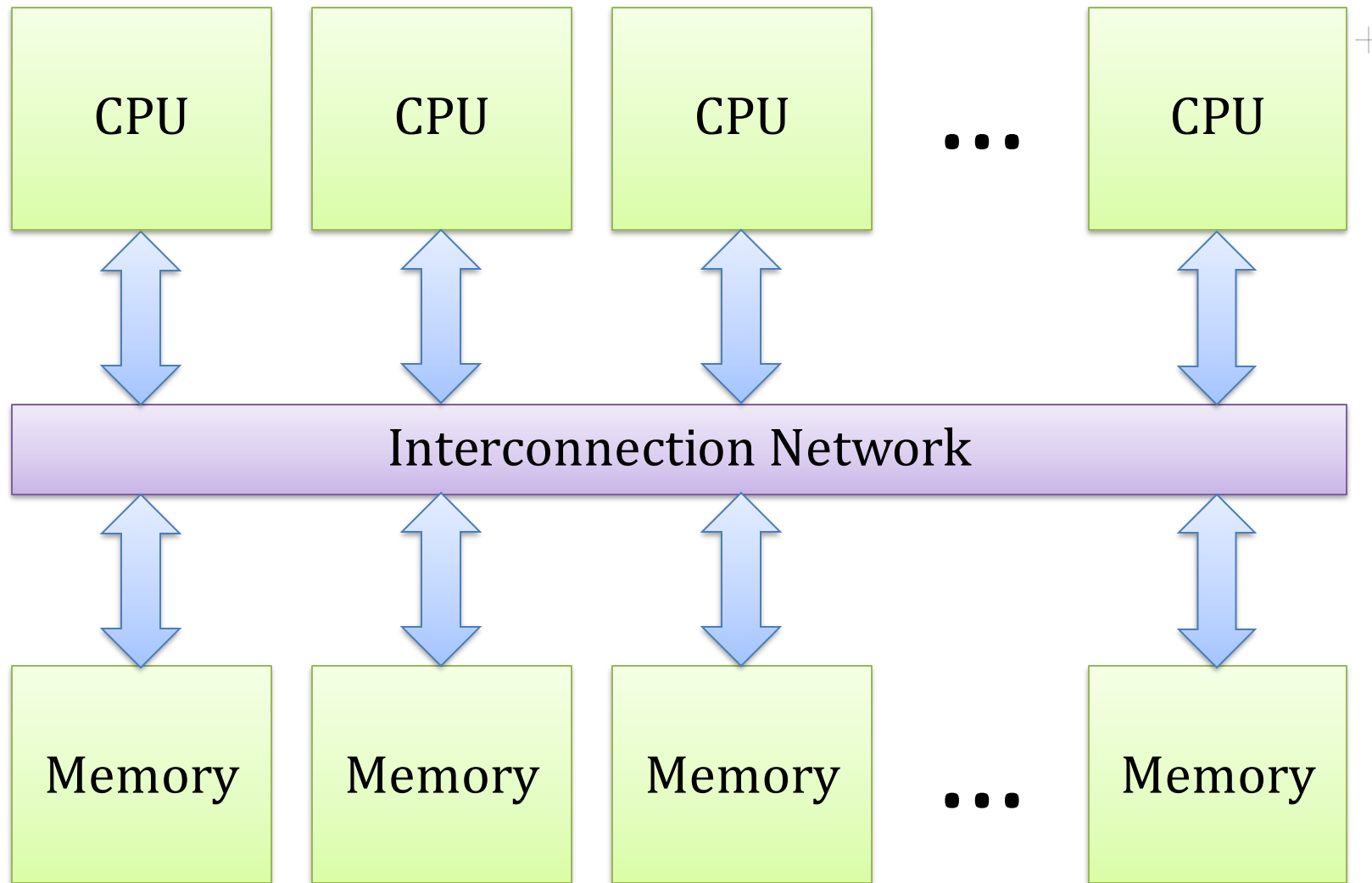
$$S_p = \frac{p}{1 + \frac{p \log p}{n}}$$

$$E_p = \frac{1}{1 + \frac{p \log p}{n}}$$

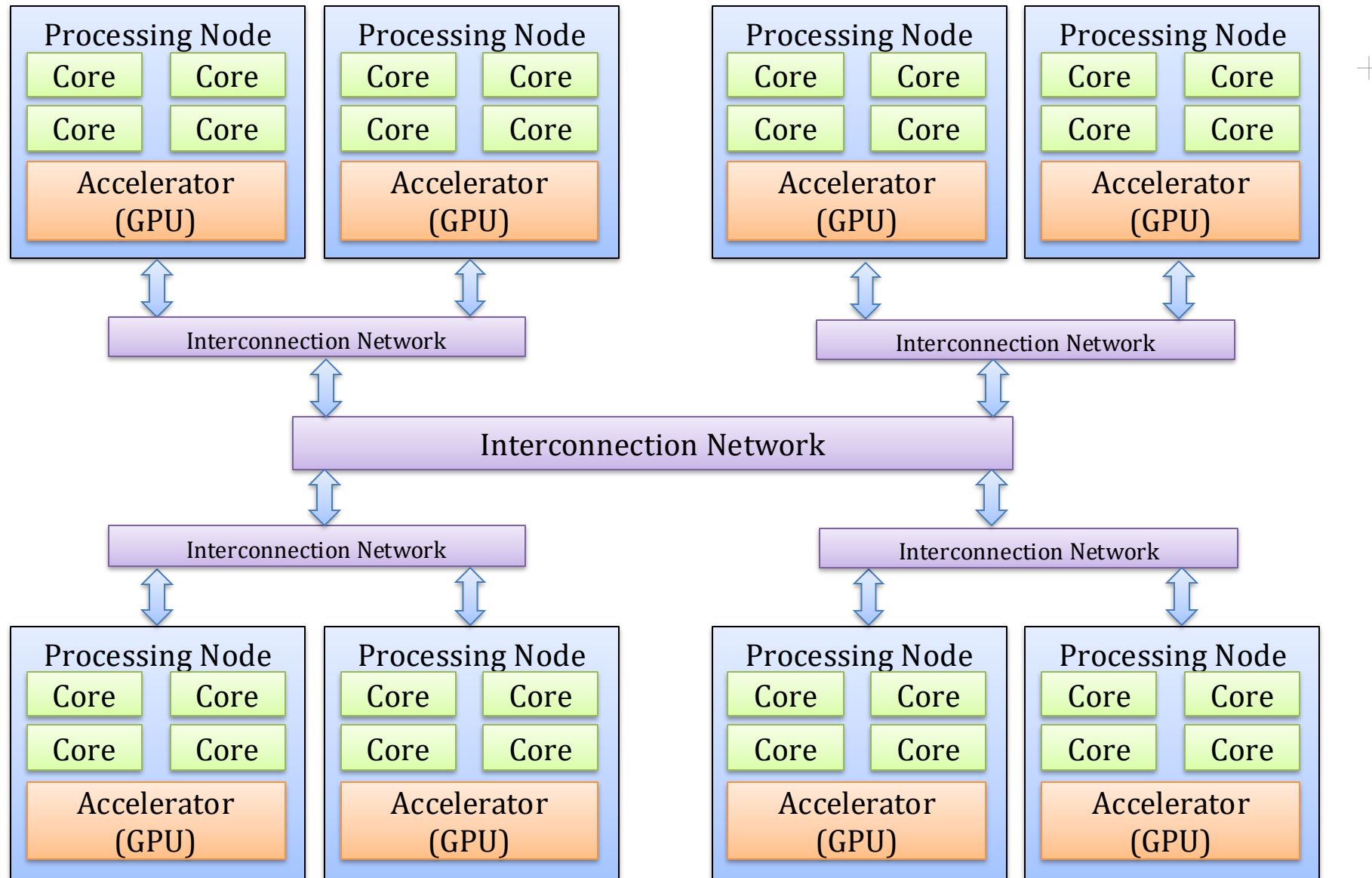
Distributed Memory Multiprocessor



Shared Memory Multiprocessor



Typical Multiprocessor



+

Programming

- Message passing: MPI
- Shared memory: POSIX threads
- Directive based: OpenMP
- Architecture specific: CUDA for GPUs



Amdahl's Law

- The limits of a parallel computation with a serial fraction s ($0 \leq s \leq 1$) are:

$$T_p = sT_1 + (1-s)\frac{T_1}{p} \qquad S_p = \frac{p}{sp + (1-s)} \qquad E_p = \frac{1}{sp + (1-s)}$$

- With increase in the number of processors p :

$$S_p \rightarrow \frac{1}{s} \qquad E_p \rightarrow 0$$

Amdahl's Law ...

